

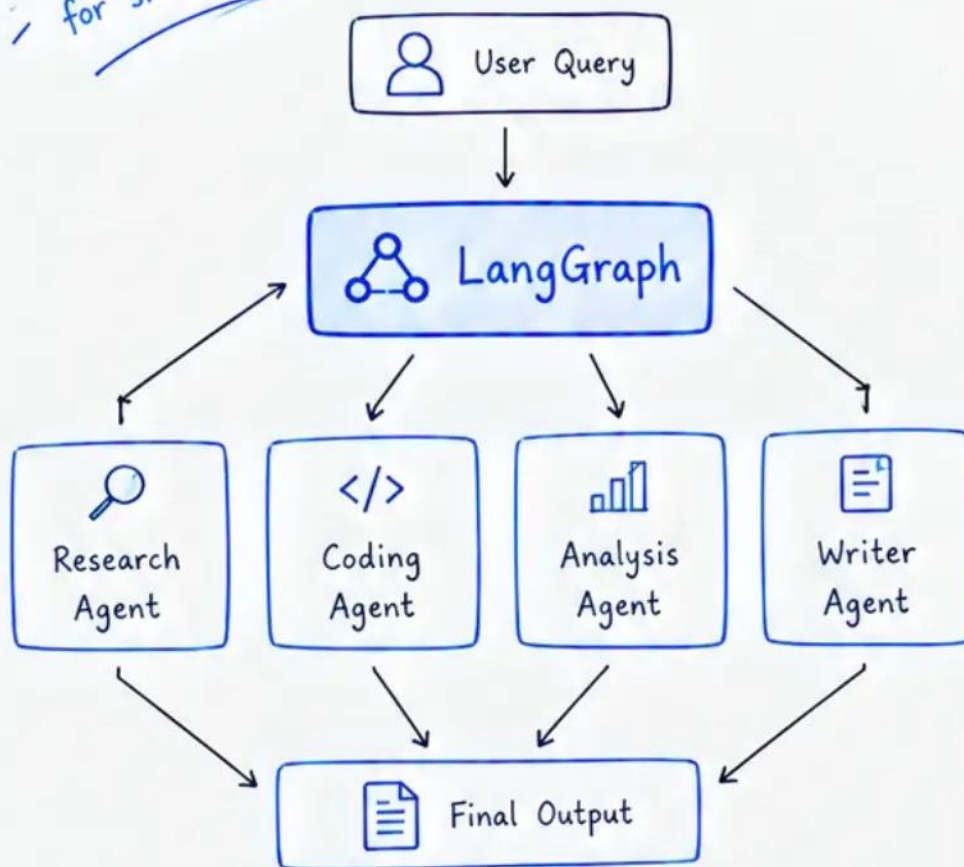


LangGraph

Multi-Agent Framework for LLMs

Build → Coordinate → Solve

Multiple agents
Working together
for smarter solutions



Subtopics Covered

- ① What is LangGraph?
- ② Graph Structure & Basics
- ③ Message Passing
- ④ Agent Nodes
- ⑤ Tools & Tool Calling
- ⑥ Memory & Persistence
- ⑦ Human-in-the-Loop
- ⑧ Advanced Graph Patterns
- ⑨ LangGraph Debugging & Observability
- ⑩ Multi-Agent Systems
- ⑪ Tool Calling & Agent Workflows
- ⑫ Error Handling & Reliability
- ⑬ Production Deployment
- ⑭ Use Cases & Real-world Apps
- ⑮ Best Practices & Tips

From simple chains
to complex multi-agent systems
— LangGraph makes it possible.

Better Agents
→ Better Workflows
→ Better AI

SUBTOPIC: Introduction to LangGraph

1. What is LangGraph?

- LangGraph is a framework for building **stateful, multi-actor** applications using graphs.
- It is an extension of LangChain, designed to handle complex workflows with better control, memory and flexibility.
- LangGraph lets you define **nodes** (LLM or tools), connect them with edges and manage the flow using a graph structure.
- It is especially useful for building agentic systems that need multiple steps, decision making and long-term memory.

☆ In short: LangGraph = Graphs + Agents + Memory
= More control, more complex workflows.

3. LangGraph vs LangChain

Feature	LangChain	LangGraph
Focus	Single / linear chains	Graph-based workflows
Workflow	Sequential	Parallel, conditional, cyclic (graph)
State	Limited	Explicit state management
Memory	Simple memory	Long-term & custom memory
Use case	Basic agent setups, RAG, tools	Complex agents, multi-agent systems, human-in-the-loop
Control	Less control	Full control over flow
Complexity	Easier to learn	Slightly complex

☆ LangChain is great for simple use cases, LangGraph is better for complex, stateful, multi-step agentic workflows.

5. Core Concepts

(a) Nodes

- Represent a step in the workflow.
- Can be an LLM, tool or a custom function.
- Each node takes input from the graph state and returns updated state.

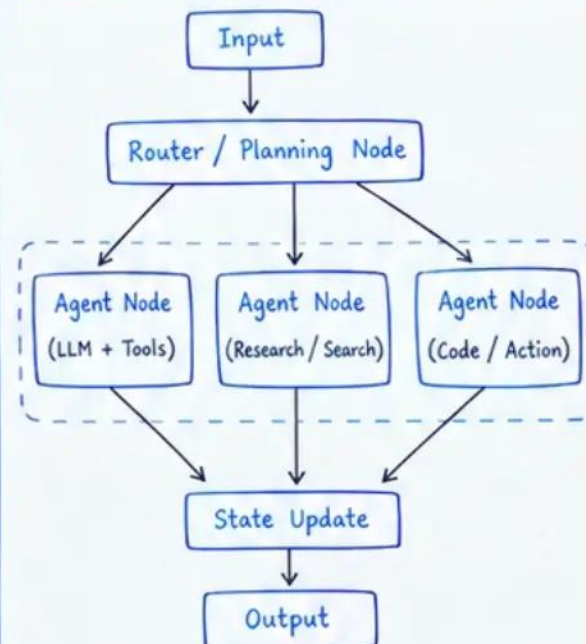
(b) Edges

- Connect nodes and define the flow.
- Can be conditional or unconditional.
- Control which node runs next.

2. Why LangGraph?

- Handles complex, multi-step workflows (not just linear).
- Supports stateful conversations and long-term memory.
- Allows parallel **execution** and conditional logic.
- Built on LangChain, so you get the same tools + more control.
- Ideal for building real **agentic systems** (e.g., research agents, coding agents, multi-agent teams).

4. Graph-based Agent Architecture



Graph allows:

- Parallel execution
- Conditional routing
- State management
- Loops / cycles

Example: Multi-agent research system

- Router decides which agent to call.
- Agents work in parallel or sequence.
- State is updated after each step.
- Final output is generated.

6. Additional Notes

- LangGraph uses a graph structure where nodes are LLM calls, tools or custom functions.
- The graph can be **static** (pre-defined) or **dynamic** (decided at runtime).
- It supports **persistence**, so the agent can remember past steps.
- Useful for workflows like research, coding, customer support, data analysis, etc.

☆ LangGraph gives you the power of a graph + the flexibility of LLMs, making it ideal for building advanced, stateful agent systems.

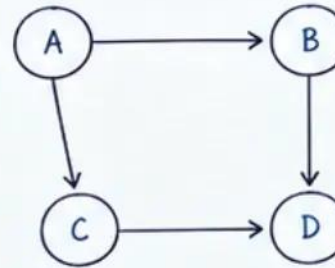
SUBTOPIC: LangGraph Core Concepts

1. Graphs

- A graph is a collection of **nodes** and **edges** that represent relationships between them.
- In LangGraph, the graph represents the **workflow** of an AI agent or multi-agent system.
- It helps in defining how different steps (nodes) are connected and executed.

★ Think of it as a flowchart, but more flexible and powerful.

Example: Simple Graph



Here:

- A, B, C, D → **nodes**
- Arrows → **edges**
- Shows possible paths for execution.

2. Nodes

- Nodes are the **individual** steps or components in the graph.
- Each node can perform a specific task, such as:
 - Calling an LLM
 - Running a tool
 - Processing data
 - Making a decision

★ Nodes can be simple functions or complex chains (like agent calls).

Types of Nodes

Type	Description
Agent Node	Runs an AI agent (LLM + tools).
Tool Node	Executes a specific tool or function.
Condition Node	Makes decisions based on conditions (e.g., if-else).
Custom Node	User-defined logic (data processing, API calls, etc.).

Example Node (Agent)

```

def agent_node(state):
    response = llm.invoke(
        state["messages"]
    )
    return {"messages": response}
  
```

- The node takes the current state and returns an updated state.

3. Edges

- Edges define the **flow** between nodes.
- They determine which node runs next.
- Can be:
 - Direct (fixed path)
 - Conditional (based on state/logic)

Example:

```

A → B (direct edge)
B → C (direct edge)
C → D (direct edge)
  
```

4. State

- State** is the **shared** data that flows through the graph.
- It keeps track of information like:
 - Messages
 - Intermediate results
 - User input / context

★ Each node can read and update the state.

State Example

```

state = {
    "messages": [],
    "user_input": "",
    "context": {}
}
  
```

5. Start and End

- Start**: The entry point of the graph.
- End**: The exit point where execution stops.
- LangGraph uses special nodes to mark the beginning and end of the workflow.

Visual Representation



- START and END are predefined special nodes in LangGraph.

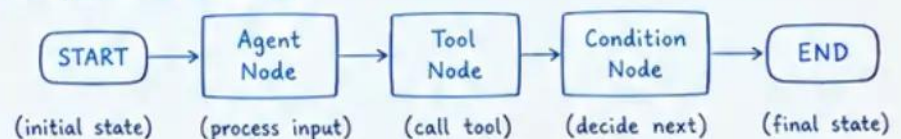
★ Key Takeaways

- Graphs define the structure.
- Nodes are the building blocks.
- Edges connect the nodes.
- State carries information.
- Start and End mark the boundaries.
- Execution flow** decides the order of operations.

6. Graph Execution Flow

- Start**: Execution begins at the **START** node.
- Traverse**: Each node performs its task and updates the state.
- Follow edges**: Move to the next node based on the edge rules (direct or conditional).
- Repeat**: Continue until the **END** node is reached.

Simple Flow Example



★ Why it matters?

These core concepts allow LangGraph to build flexible, stateful and powerful agent workflows, making it easier to create complex multi-step AI applications.

SUBTOPIC: State Management

1. Defining State

- **State** is a shared data object that flows through the graph.
- It stores information (messages, intermediate results, user inputs, etc.) that is updated by different nodes.
- Each node takes the current **state** as input and returns a **partial** or full update to the state.
- State makes LangGraph **stateful** and enables complex multi-step workflows.

☆ In short: State = memory of the graph.

2. State Schemas

- A **state schema** defines the **structure** (keys and types) of the state.
- Helps in **type safety**, better code clarity and validation.
- Can be defined using **TypedDict** (recommended) or Pydantic model.
- The schema acts like a **contract** for what data is stored in the state.

Example schema (using TypedDict)

```
from typing import TypedDict, List
class AgentState(TypedDict):
    messages: List[dict]
    user_input: str
    intermediate_steps: List[str]
    final_answer: str
    context: dict
```

- Each key represents a piece of information shared across nodes.

3. TypedDict

- TypedDict (from Python's **typing** module) is commonly used to define the state schema in LangGraph.
- Provides type hints for each field.
- Lightweight and easy to use.

```
from typing import TypedDict
class MyState(TypedDict):
    query: str
    messages: list
    result: str
    metadata: dict
```

Note: You can also use Pydantic BaseModel for more validation, especially in production.

4. State Updates

- Each node returns a dictionary containing the fields to update.
- LangGraph automatically merges the updates with the existing state.
- You only need to return the changed fields, not the entire state.

Example: Node function

```
def retriever_node(state: AgentState):
    docs = retrieve_docs(state["user_input"])
    return {"context": docs,
            "intermediate_steps": state["intermediate_steps"] + ["retrieved docs"]}
```

→ Only return the fields you want to update.

5. State Reducers

- **State reducers** define how updates from nodes are merged into the existing state.
- Useful when multiple nodes update the **same** key (e.g., appending messages).
- By default, LangGraph uses a simple overwrite, but you can specify **custom reducers** for specific fields.

Common reducers:

Reducer	Description	Use case
overwrite (default)	Replace old value	Single writer
append	Append to a list	Chat messages, logs
extend	Extend a list	Multiple results
merge_dict	Merge dictionaries	Aggregating info
custom reducer	User-defined logic	Special cases

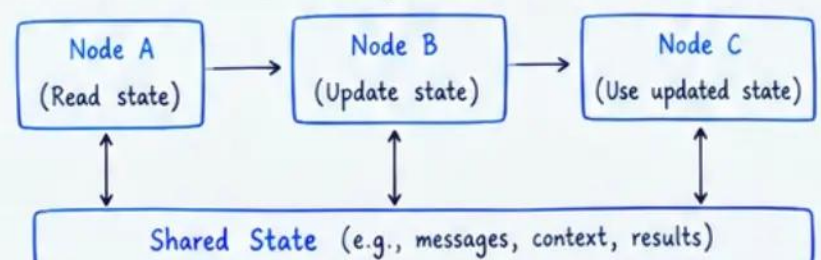
Example: Using a reducer (append for messages)

```
from langgraph.graph import StateGraph
from langgraph.graph.state import add_messages
graph = StateGraph(AgentState, reducers={
    "messages": add_messages
})
```

Here, messages from different nodes will be automatically appended to the list instead of overwritten.

6. Shared State Between Nodes

- The **state** is shared **across all nodes** in the graph.
- Each node can read from and update the same state.
- This enables communication between nodes without explicit passing of data.
- Useful for multi-step reasoning, tool use, and memory retention.



SUBTOPIC: Nodes in LangGraph

1. Creating Nodes

- Nodes are created using functions (usually Python functions) or LangChain runnables.
- Each node performs a specific task and returns an output that becomes part of the graph's state or passed to the next node.
- You can create nodes using a simple function or a `@tool / Runnable` if needed.

Example:

```
def process_input(state):
    # do some work
    result = ...
    return {"messages": [result]}

# or using a tool
from langgraph.prebuilt import tool
@tool
def search_tool(query: str):
    return f"Search results for: {query}"
```

A node can be a simple function, a tool or a Runnable.

2. Node Functions

- Node functions define the logic to be executed at that step.
- They take the current state as input and return updated state (or partial update).
- Can contain LLM calls, tool calls, conditions, or custom logic.

Common patterns:

- LLM call (chat model)
- Tool call (search, calculator, etc.)
- Data processing
- Conditional logic

Example:

```
def node_func(state):
    user_msg = state["messages"][-1]
    response = llm.invoke(user_msg)
    return {"messages": [response]}
```

★ Node functions are pure functions (no side effects outside the state).

3. Input / Output

- Input: The current state (dictionary) is passed to the node function.
- Output: The node returns a dictionary (partial or full state update).
- The output is merged with the existing state using reducers (if needed).

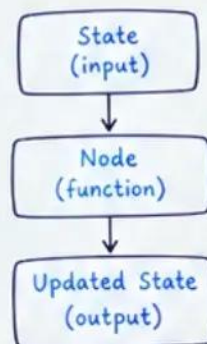
Example:

Input state:

- `{"messages": ["Hi"], "user_input": "..."}`

Node output:

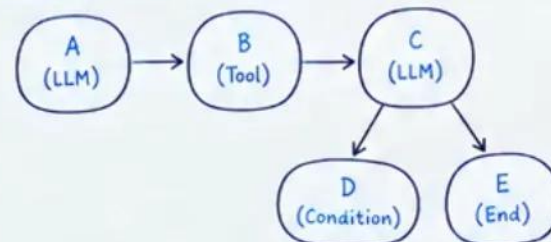
- `{"messages": ["Hello!"], "next": "tools"}`



4. Multiple Nodes

- A graph can have multiple nodes, each doing a different task.
- Nodes are connected using edges to define the flow.
- This allows complex workflows with multiple steps and decision points.

Example: Simple multi-node graph



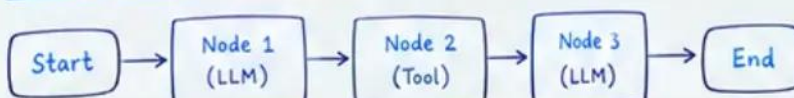
Each node can:

- call an LLM
- use a tool
- process data
- update state

5. Node Execution

- Nodes are executed in the order defined by the graph (or based on conditions).
- Execution can be sequential, parallel or conditional.
- LangGraph handles the state passing and transitions automatically.

Execution flow (sequential example):



Key points:

- Each node runs with the current state.
- The returned state is merged using reducers.
- The next node's decided based on edges / conditions.

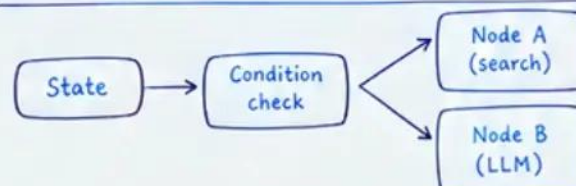
6. Dynamic Node Behavior

- Nodes can change behavior based on the current state.
- You can use conditional edges, state checks or custom logic to decide the next node.
- Enables dynamic and adaptive workflows.

Example: Conditional routing

```
def route_node(state):
    if state["user_input"].lower() == "search":
        return "search_node"
    else:
        return "llm_node"
```

Use conditional edges or custom functions to make the graph adaptive.

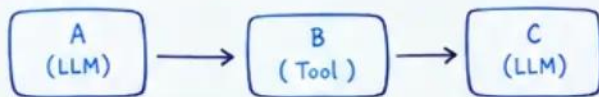


SUBTOPIC: Edges & Control Flow

1. Normal Edges

- Normal edges are the default way to move from one node to another.
- They represent a direct, unconditional flow in the graph.
- Used when the next step is always the same regardless of input or state.

Example: Simple linear flow

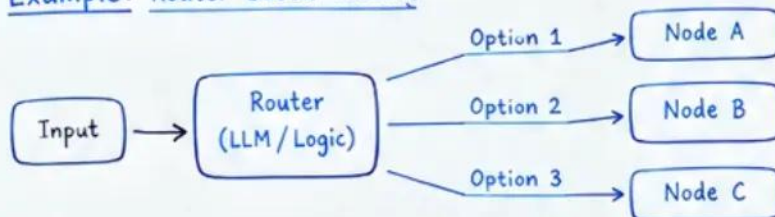


Here, after node A, the flow always goes to node B, then to node C.

3. Routing

- Routing is a special type of conditional edge where the next node is chosen from multiple options.
- Often used in a router node or a decision function.
- Allows flexible and dynamic control over the flow.

Example: Router based routing

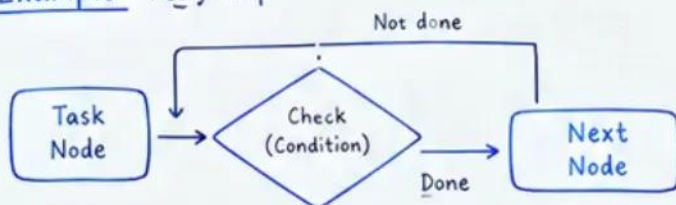


The router selects one of the possible nodes based on the input or state.

5. Loops

- Loops are used if an action needs to be repeated until a condition is met.
- Can be implemented using conditional edges that point back to a previous node.
- Useful for iterative reasoning, retries, or multi-step processing.

Example: Retry loop



The flow keeps repeating until the condition is satisfied.

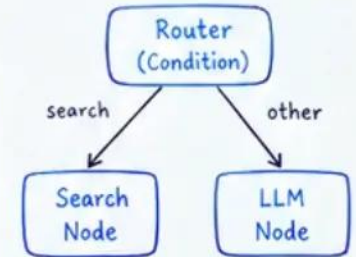
2. Conditional Edges

- Conditional edges are used when the next node depends on a condition or state value.
- You define a condition function that returns the name of the next node.
- Useful for branching logic (e.g., if-else, decisions).

Example:

```

def route(state):
    if state["query_type"] == "search":
        return "search_node"
    else:
        return "llm_node"
  
```

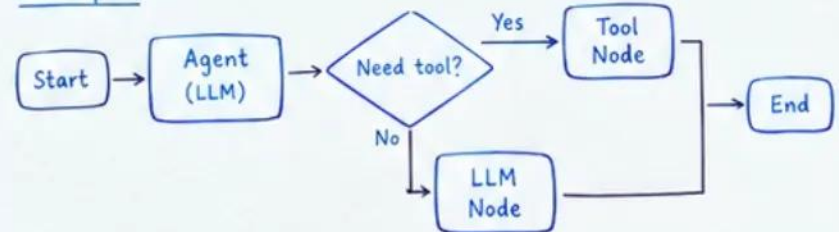


The condition decides which node to execute next.

4. Dynamic Paths

- Dynamic paths allow the graph to determine the next steps at runtime.
- The path can change based on the data, state, or external input.
- Useful for routing through various tool usage, or adaptive workflows.

Example:

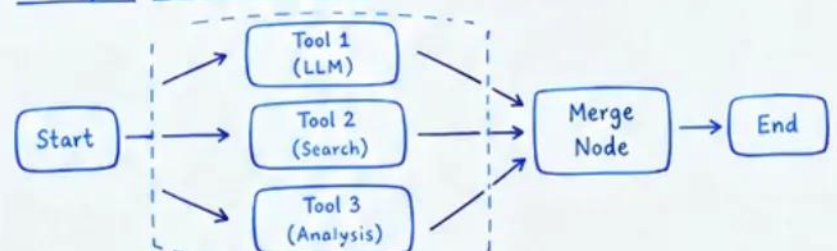


The path is decided dynamically during execution.

6. Parallel Execution

- Parallel execution allows multiple nodes to run at the same time.
- Useful for independent tasks (e.g., multiple tools, data fetch, analysis).
- Improves efficiency and reduces overall time.

Example: Parallel tool calls



All three nodes run in parallel, and the results are merged before moving to the next step.

★ Key Takeaways:

- Edges define how the flow moves between nodes.
- Conditional edges, routing and dynamic paths enable decision making.
- Loops help with iteration and retries.
- Parallel execution improves efficiency and speed.

SUBTOPIC: Building Your First LangGraph

1. Creating a StateGraph

- StateGraph is the main class used to build a graph with state and nodes.
- It helps define the state schema and the flow between nodes.
- You can specify the state type using a TypedDict or a custom class.

Example:

```
from typing import TypedDict
from langgraph.graph import StateGraph

class MyState(TypedDict):
    messages: list[str]

graph = StateGraph(MyState)
```

StateGraph is initialized with the state schema.

2. Adding nodes

- Nodes are the units of work (functions) in your graph.
- Each node takes the current state as input and returns an updated state.
- You can add multiple nodes using add_node().

Example:

```
def chatbot_node(state: MyState):
    response = llm.invoke(state["messages"])
    return {"messages": state["messages"] + [response]}

graph.add_node("chatbot", chatbot_node)
```

Node name must be unique in the graph.

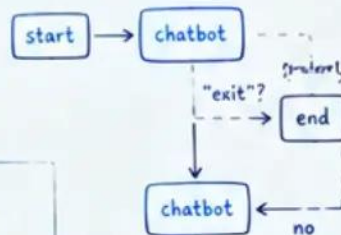
3. Connecting nodes

- Nodes are connected using edges.
- You can define a simple linear flow or conditional edges based on logic.
- Use add_edge() for normal flow and add_conditional_edges() for branching, for branching.

Example (Normal edge):

```
graph.add_edge("start", "chatbot")
graph.add_edge("chatbot", "end")
```

Example Flow:



Example (Conditional edge):

```
def route(state: MyState):
    if "exit" in state["messages"][-1]:
        return "end"
    return "chatbot"

graph.add_conditional_edges("chatbot", route)
```

4. Compiling graphs

- After adding nodes and edges, you need to compile the graph.
- The compile() method builds the internal structure and prepares it for execution.
- You can also add a checkpoint for memory (optional).

Example:

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
app = graph.compile(checkpointer=memory)
```

Without compile(), the graph cannot be executed.

5. Invoking a graph

- Use invoke() to run the graph with an initial input state state.
- The graph processes the nodes in order and returns the final state.
- You can also stream the output using stream() for real-time results.

Example:

```
initial_state = {"messages": ["Hello"]}
result = app.invoke(initial_state)
print(result["messages"])
```

invoke() returns the final state after execution.

6. Basic chatbot hdx example

- This example shows a simple chatbot using LangGraph with a single node.

Graph Flow:



Full Code:

```
from typing import TypedDict
from langgraph.graph import StateGraph

class State(TypedDict):
    messages: list[str]

def chatbot_node(state: State):
    response = "You said: " + state["messages"][-1]
    return {"messages": state["messages"] + [response]}

graph = StateGraph(State)
graph.add_node("chatbot", chatbot_node)
graph.add_edge("start", "chatbot")
graph.add_edge("chatbot", "end")
app = graph.compile()

result = app.invoke({"messages": ["Hello! How are you?"]})
print(result["messages"])
```

This is a minimal example. You can add more nodes, logic and memory as needed.

★ Quick Revision

- Use StateGraph to create the graph.
- Add nodes with add_node().
- Connect nodes using add_edge() or add_conditional_edges().
- Compile the graph before execution.
- Use invoke() or stream() to run the graph.
- Each node returns an updated state.

★ Key Takeaway

Building a LangGraph involves defining the state, adding nodes, connecting them, compiling the graph, and invoking it with an input state.

SUBTOPIC: LangGraph with LLMs

1. Connecting LLMs

- LangGraph can connect to different LLM providers using integrations.
- Supports OpenAI, Anthropic, HuggingFace and more.
- You can configure the LLM with API keys, model name, temperature, etc.

Example: Connecting OpenAI model

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o",
    temperature=0.7,
    api_key="your-api-key"
)
```

Other providers:

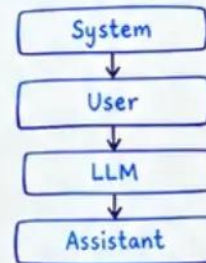
- Anthropic
- HuggingFace
- Ollama
- Google
- etc.

2. Chat Models

- Chat models are LLMs designed for conversation.
- They use a message-based format (role + content).
- Common roles: system, user, assistant, tool.

Example: Chat model with messages

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is LangGraph?"}
]
```



Message roles:

- **system** → sets behavior
- **user** → user input
- **assistant** → model response
- **tool** → tool/function output

3. Messages

- Messages are the basic unit of communication with the chat model.
- Each message has a role and content.
- Can include additional fields like name, tool_calls, etc.

Example: Message format

```
{
  "role": "user",
  "content": "Hello!"
}
```

Common fields:

- role
- content
- name
- tool_calls
- function_call

4. Prompt Handling

- LangGraph helps manage prompts across nodes.
- You can use templates, dynamic variables, and system prompts.
- Supports fixing typed outputs and function calling.

Example: Using prompt template

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a {role} assistant."),
    ("user", "{input}")
])
```

Key features:

- Templates
- Variables
- System prompts
- Structured output
- Function calling

5. LLM Nodes

- LLM nodes wrap the chat model and handle prompt + response logic.
- Can be added to the graph like any other node.
- Supports custom logic, tools and memory.

Example: LLM node function

```
def call_llm(state):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

llm_node = StateNode(call_llm) # or custom wrapper
```

LLM node can:

- call model
- use tools
- update state
- add memory
- handle errors

6. Streaming Responses

- LangGraph supports streaming responses for real-time output.
- Useful for receiving large outputs cut at.
- You can stream tokens or message chunks.

Example: Streaming with LangGraph

```
for event in graph.stream(input_data):
    if "messages" in event:
        print(event["messages"][0]["content"], end="")
```

Benefits:

- Real-time output
- Better UX
- Handles long responses
- More interactive

☆ Complete Flow Example



Flow:

1. User sends input message.
2. Prompt is prepared and handled.
3. LLM node calls the chat model.
4. Response is streamed back to the user.

Key Takeaways:

- Connect LLMs using integrations.
- Use chat models with message format.
- Handle prompts with templates and variables.
- Wrap LLMs in nodes for graph execution.
- Use streaming for real-time responses.

Real-world Use Cases:

- AI chat assistants
- Document Q&A systems
- Customer support bots
- Research agents
- Multi-agent systems

SUBTOPIC: Memory & Persistence

1. Short-term Memory

- Short-term memory stores information only for the current conversation / execution.
- It is usually kept in the state object while the graph is running.
- Useful for maintaining context between nodes in a single run.

Example: Storing recent user input in state

```
def update_state(state, user_input):
    state["messages"].append({
        "role": "user",
        "content": user_input
    })
    return state
```

Key points:

- Temporary
- Lost after run ends
- Used for current context

2. Long-term Memory

- Long-term memory persists across multiple runs and conversations.
- It can be stored in external systems like databases, vector stores, or files.
- Used to remember user preferences, past interactions, or domain knowledge.

Example: Using a simple database to store user info

```
def save_user_memory(user_id, data):
    db.store(user_id, data)

def load_user_memory(user_id):
    return db.get(user_id)
```

Use cases:

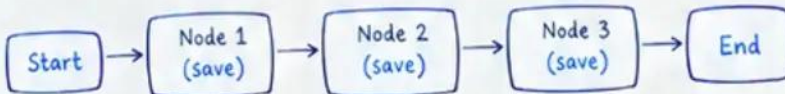
- User preferences
- Past conversations
- Domain knowledge
- Personalization

3. Checkpoints

- Checkpoints save the state of the graph and certain points in execution.
- If the execution fails, it can be resumed from the last checkpointing in the checkpointing, useful for cost savings.
- Useful for running workflows and fault tolerance.

Example: Saving checkpoint after using that node

```
graph = builder.compile(checkpointer=MemorySaver())
# during execution
graph.invoke(input_data, Config={"configurable": {"checkpoint": True}})
```



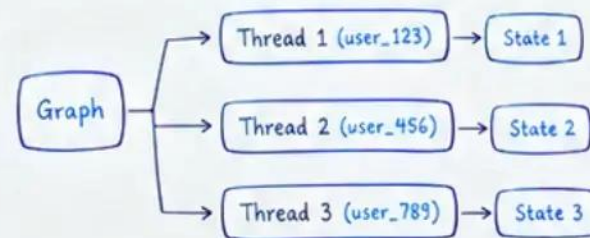
If interrupted, execution can resume from the last saved checkpoint.

4. Threads

- Threads allow multiple independent conversations or workflows to run separately.
- Each thread has its own state and memory.
- Helps in managing parallel users or different tasks.

Example: Running multiple threads

```
thread_config = {"thread_id": "user_123"}
result = graph.invoke(input_data, config=thread_config)
```



Benefits:

- Isolated state
- Parallel runs
- Better scalability

5. Persistent State

- Persistent state is stored outside the graph thread current run and the code survives restarts.
- It can be saved in databases, vector stores, or cloud storage.
- Used for long-term workflows and production systems.

Example: Saving state to a database

```
def save_persistent_state(thread_id, state):
    db.save(thread_id, state)

def load_persistent_state(thread_id):
    return db.load(thread_id)
```

Storage options:

- Database
- Vector store
- File system
- Cloud storage

★ Useful for production apps and long workflows.

6. Conversation History

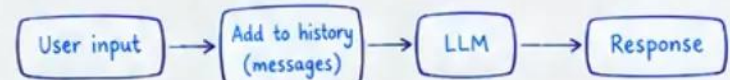
- Conversation history stores the sequence of messages in a chat.
- It can be retrieved and passed to the LLM for context.
- Helps in maintaining coherent and relevant responses.

Example: Adding Conversation history to prompt

```
history = state["messages"]
response = llm.invoke({
    "messages": history + [new_message]
})
```

Features:

- Ordered messages
- Role (user/assistant)
- Can be stored in memory or DB



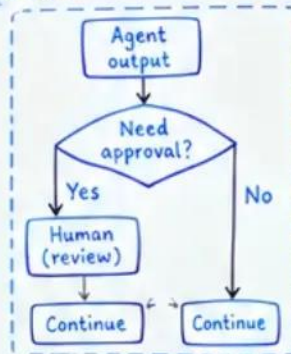
SUBTOPIC: Human-in-the-Loop

1. Human Approval

- Human approval allows a person to review and approve, reject, or modify the output of an agent before the next step is executed.
- It is useful for sensitive tasks like data access, code execution, or critical decisions.
- You can pause the graph and wait for user input using interrupts or approval nodes.

Example: Ask for approval before execution

```
def approve_node(state):
    if state["action"] == "delete":
        return {"needs_approval": True}
    return {"needs_approval": False}
```



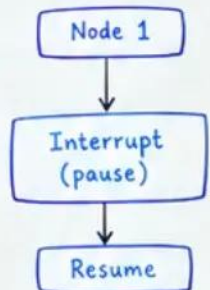
2. Interrupts

- Interrupts allow you to pause the graph at a specific point and wait for external input.
- You can specify the interrupt point using a node or condition.
- The graph will stop and return control to the caller until it is resumed.

Example: Pause at a specific node

```
from langgraph.types import Interrupt

def check_node(state):
    # pause and wait for user input
    return Interrupt(value="__human_input__")
```



4. Editing State

- Humans can modify the state before resuming execution.
- This helps correct errors, update information, or add additional context.
- The updated state is used by the next node.

Example: Update state with user edits

```
new_state = {**state, "query": updated_query}
graph.invoke(new_state, resume=True)
```

What can be edited?

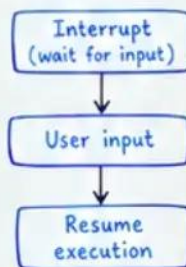
- Messages
- Variables
- Tool results
- User preferences
- Custom fields

3. Resume Execution

- After a human provides input, the graph can resume from where it was paused.
- The state is preserved, execution continues without losing context.
- Useful for multi-step approvals or long-running tasks.

Example: Resume after user approval

```
Graph.invoke({"messages": [...],
             "resume": True})
```



5. Human Approval Workflows

- You can build custom workflows with multiple approval steps (e.g. manager → admin → final).
- Each step can have its own conditions and logic.
- Useful in enterprise workflows and compliance tasks.

Example: Multi-step approval flow

```
workflow = [
    {"role": "user"},
    {"role": "manager"},
    {"role": "admin"}
]
```

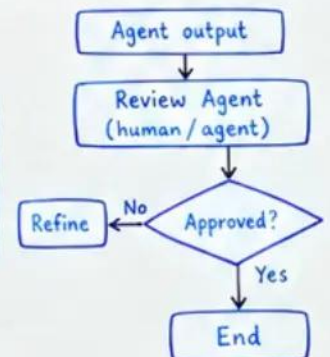


6. Review Based Agents

- Review agents use a human (or another agent) to evaluate the final output.
- They can check for accuracy, safety, and quality.
- Useful for research, content generation, and code review.

Example: Review agent workflow

```
def review_node(state):
    # send output for review
    return {"pending_review": True}
```



★ Key Takeaways:

- Human approval adds safety and control.
- Interrupts pause execution at specific points.
- Resume execution continues with preserved state.
- You can edit state before resuming.
- Approval workflows support multi-step review.
- Review-based agents ensure quality and correctness.

SUBTOPIC: Tool Calling & Agent Workflows

1. Tools in LangGraph

- Tools are EXTERNAL functions or APIs that the agent can call to perform real-world actions.
- They help the LLM go BEYOND text generation and interact with external systems (e.g. search, calculator, database, etc).
- In LangGraph, tools are integrated with nodes and used through tool nodes or function calling.

Example: Tools can be used for web search, fetching data, running code, etc.

```
from langgraph.prebuilt import Tool
tools = [
    Tool(name="search", func=search_api),
    Tool(name="calculator", func=calc_api)
]
```

Common tools:

- Search
- Calculator
- Database
- Code executor
- File system

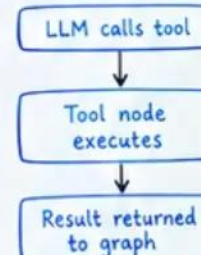
2. Tool Nodes

- Tool nodes are special nodes in LangGraph that execute a tool when called by the LLM.
- They take the tool name and input from the model, run the tool, and return the result to the graph.
- Used with conditional EDGES to decide the next step.

Example: Adding a tool node

```
def search_tool(state):
    query = state["query"]
    result = search_api(query)
    return {"tool_result": result}
```

Tool node flow:



3. Function Calling

- FUNCTION CALLING lets the LLM decide when to call a tool and what arguments to pass.
- The Model returns a structured output (tool name + args) which is then executed by the tool node.
- In LangGraph, this is handled automatically using tool binding and structured output.

Example: Function calling with OpenAI

```
llm = ChatOpenAI(temperature=0)
llm = llm.bind_tools(tools)
response = llm.invoke(messages)
# response.tool_calls contains tool name + args
```

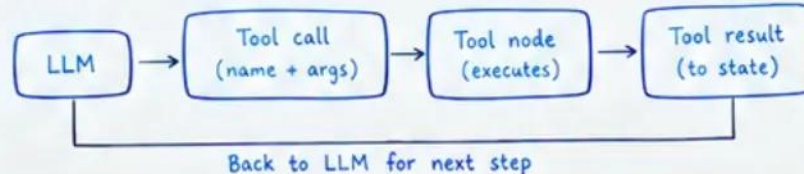
Model output:

```
{
  "tool_calls": [
    {
      "name": "search",
      "args": {"query": "AI"}
    }
  ]
}
```

4. Tool Execution

- After the LLM decides to call a tool, the tool node executes the function with provided arguments.
- The result is stored in the state and passed back to the model for further reasoning.
- This enables iterative tool use and better responses.

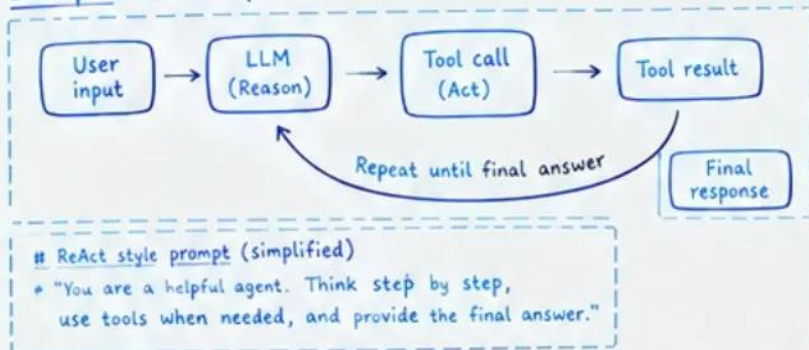
Example: Execution flow



5. ReAct-style Agents

- ReAct (Reason + Act) combines reasoning with tool use.
- The Model thinks about the problem, decides which tool to use, executes it, and then reasons again.
- Works well for complex tasks requiring multiple steps and tool interactions.

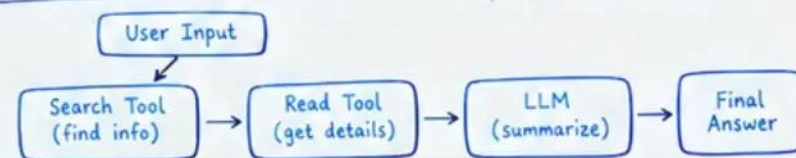
Example: ReAct loop



6. Multi-step Tool Workflows

- FOR complex tasks, multiple tools may be used in sequence or in parallel.
- LANGGRAPH MANAGES the flow using nodes, edges and conditional logic.
- Each step updates the state and moves to the next node.

Example: Multi-step workflow (search -> read -> summarize)



Use cases:

- Research assistants
- Travel planners
- Data analysis agents
- Complex reasoning tasks

Key points:

- Use conditional edges
- Store intermediate results
- Handle errors
- Keep state updated

— Key Takeaways —

- TOOLS extend the agent's capabilities.
- Use tool nodes + function calling for execution.
- ReAct enables reasoning + tool use.

- Multi-step workflows handle complex tasks.
- Manage state and errors for reliable execution.
- Combine tools, nodes and conditions for best results.

SUBTOPIC: Multi-Agent Systems

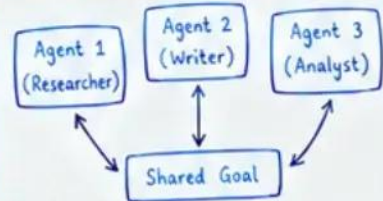
1. What are Multi-Agent Systems?

- Multi-agent systems (MAS) are systems where multiple autonomous agents work together to achieve a common goal.
- Each agent has its own role, tools, memory and decision-making ability.
- Agents can communicate, share information and coordinate with each other.

Example: A research assistant team with different agents (researcher, writer, analyst) working together.

Key characteristics:

- Multiple autonomous agents
- Communication & coordination
- Shared or individual goals
- Flexible and scalable



2. Agent-to-Agent Communication

- Agents exchange information using messages.
- Communication can be direct (pairwise) or via a mediator (e.g. supervisor).
- Messages contain content, sender, receiver and optional metadata.

Example: Message format

```

{
  "sender": "agent_1",
  "receiver": "agent_2",
  "content": "Need data on latest research.",
  "metadata": {"task_id": "123"}
}
  
```

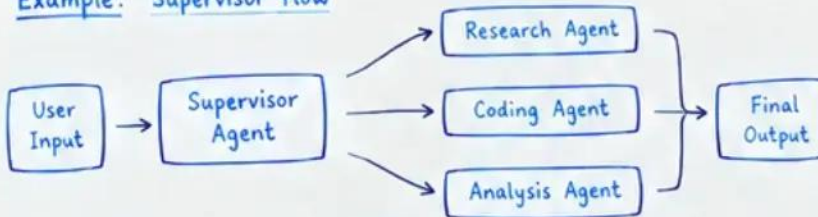
Communication types:



3. Supervisor Architecture

- Supervisor agent manages multiple worker agents.
- It assigns tasks, gets results and makes decisions.
- Helps in complex tasks where different agents have different strengths.

Example: Supervisor flow



Benefits:

- Better task management
- Avoids conflicts between agents
- Scales to multiple agents easily

4. Agent Collaboration

- Agents share information and build on each other's work.
- Can happen through shared memory, messages or tools.
- Improves accuracy and quality of results.

Example: Collaborative research



Collaboration techniques:

- Shared state / memory
- Message passing
- Tool usage
- Iterative refinement

5. Agent Routing

- Determines which agent should handle a task.
- Can be based on rules, LLM decisions or conditions.
- Helps in efficient use of resources and expertise.

Example: Routing based on task type

```

if task == "research":
    route_to(research_agent)
elif task == "code":
    route_to(coding_agent)
else:
    route_to(general_agent)
  
```

Routing methods:

- Rule-based
- LLM-based
- Conditional edges
- Tool-based

★ Key Takeaways:

- Multiple agents work together to achieve a goal.
- Communication, routing and collaboration are key.
- Supervisor architecture helps manage complex tasks.

6. Multi-Agent Workflows

- Multi-agent workflows define the sequence and flow of tasks between agents.
- Can be linear, parallel or conditional.
- Often implemented using LangGraph's edges and conditions.

Example: Simple workflow



Types of workflows:

- Linear (step by step)
- Parallel (multiple agents at once)
- Conditional (based on results)

Use cases:

- Research & analysis
- Content generation
- Code generation
- Complex problem solving

- Workflows can be linear, parallel or conditional.
- Use shared memory and tools for better coordination.
- MAS improves accuracy, efficiency and scalability.

SUBTOPIC: Advanced Graph Patterns

1. Subgraphs

- A subgraph is a reusable part of a larger graph.
- Helps in modularity and keeps the main graph clean.
- Can be compiled separately and then added to the main graph.

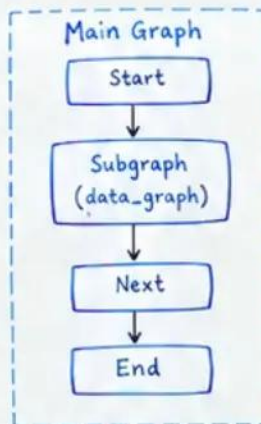
Example: Using a subgraph for data processing.

```
from langgraph.graph import StateGraph
from subgraph import data_graph

main_graph = StateGraph()
main_graph.add_node("process", data_graph)
main_graph.add_edge("process", "next")
```

Key points:

- Improves reusability
- Helps with maintainability
- Useful for complex workflows



2. Nested Workflows

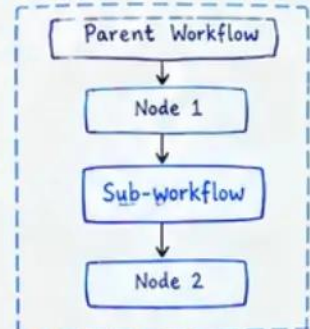
- A workflow can call another workflow inside a node.
- Useful for breaking down complex logic into smaller workflows.
- Helps in composition and reusability.

Example: Calling a sub-workflow from a node

```
def parent_workflow(state):
    result = invoke(sub_workflow,
                    {"input": state["data"]})
    return {"result": result}
```

Use cases:

- Complex multi-step tasks
- Domain-specific workflows
- Reusing existing logic



3. Parallel Branches

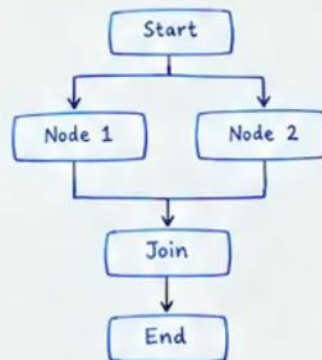
- Multiple nodes can run at the same time.
- Useful when tasks are independent.
- Reduces total execution time.

Example: Running two tasks in parallel

```
def parallel_workflow(state):
    return {
        "task1": node1(state),
        "task2": node2(state)
    }
```

Key points:

- Tasks must be independent
- Use join node to combine results
- Improves performance and efficiency



4. Map-reduce Patterns

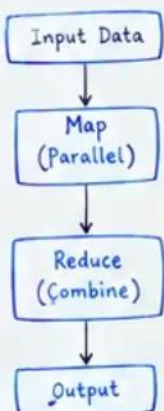
- Used to process a list of items in parallel (map).
- Results are combined using a reduce step.
- Useful for large data processing and summarization.

Example: Example: Word count

```
def map_reduce(state):
    # map: process each item
    map_result = [map_fn(item) for item in state["data"]]
    # reduce: combine results
    result = reduce_fn(map_result)
    return {"result": result}
```

Use cases:

- Data aggregation
- Summarization
- Analytics and reporting



5. Loops and Cycles

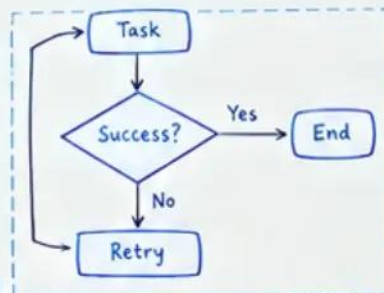
- LangGraph supports cycles using conditional edges.
- Useful for iterative tasks like retries or feedback loops.
- Must define a stopping condition to avoid infinite loops.

Example: Retry until success

```
def retry_workflow(state):
    if state["status"] == "success":
        return "end"
    else:
        return "retry"
```

Key points:

- Always define a stopping condition
- Use conditional edges for loops
- Track iterations in state

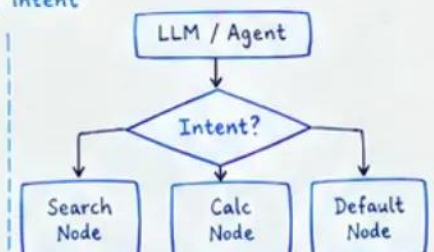


6. Dynamic Routing

- Next node is decided at runtime based on state or LLM output.
- Allows flexible and intelligent flow.
- Useful for agent systems and decision-making.

Example: Routing based on user intent

```
def route(state):
    intent = state["intent"]
    if intent == "search":
        return "search_node"
    elif intent == "calculate":
        return "calc_node"
    else:
        return "default_node"
```



Benefits:

- Flexible and adaptive
- Improves decision making
- Handles diverse user inputs

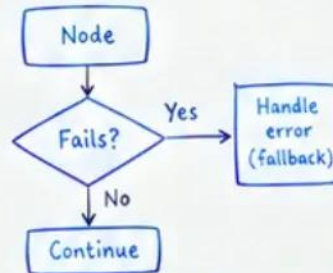
SUBTOPIC: Error Handling & Reliability

1. Handling Node Failures

- A node can fail due to exceptions, invalid input, external API errors, or unexpected conditions.
- LangGraph lets you handle failures without stopping the whole graph.
- You can define fallback logic, error outputs or retry mechanisms for failed nodes.

Example: Using try-except in a node

```
def process(state):
    try:
        result = risky_operation()
        return {"result": result}
    except Exception as e:
        return {"error": str(e)}
```

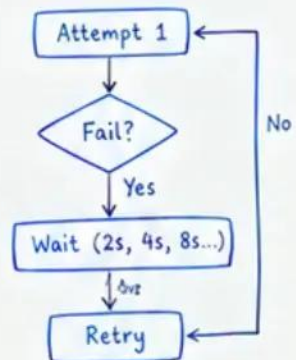


2. Retries

- Retries help recover from temporary failures like timeouts or network issues.
- You can set the number of retries and a delay between attempts.
- Use exponential backoff for better reliability.

Example: Retry with exponential backoff

```
from langgraph.retry import retry
@retry(max_attempts=3, delay=2)
def call_api(state):
    # some api call
    return result
```

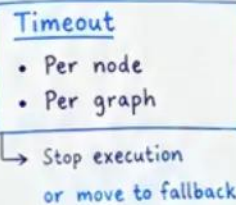


3. Timeouts

- Prevents a node from running indefinitely.
- Useful for longget, API calls or heavy computations.
- You can set timeouts for the whole graph.

Example: Node with timeout

```
def slow_task(state):
    result = long_running_function()
    return {"result": result}
```

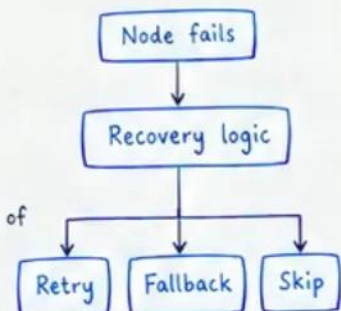


4. Error Recovery

- After a failure, you can recover by:
 - retrying the task
 - using a fallback node
 - skipping the failed step
 - or returning a default value
- Helps the graph continue working instead of failing completely.

Example: Fallback node

```
def fallback(state):
    return {"result": "default value"}
```

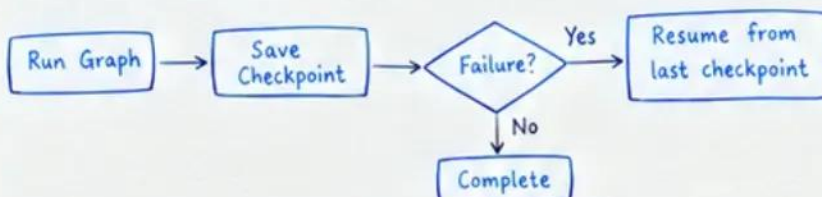


5. Checkpoint-based Recovery

- Checkpoints save the state of the graph at key points.
- If a failure occurs, you can resume from the last successful checkpoint.
- Useful for long-running workflows and complex graphs.

Example: Using checkpoint saver

```
from langgraph.checkpoint import MemorySaver
memory = MemorySaver()
graph = graph.compile(checkpointer=memory)
```



★ Checkpoints can be stored in memory, file system, or external databases.

6. Reliable Agent Execution

- Combine multiple techniques (retry, timeout, recovery, checkpointing) for robust execution.
- Use structured error handling and logging.
- Ensures agents can handle real-world failures and still complete the task.
- Improves stability, consistency and trust in production.

Example: Reliable agent setup

```
from langgraph import Graph
graph = Graph()
    .add_node("agent", agent_func, retry=3, timeout=60)
    .add_node("tool", tool_func, retry=2)
    .compile(checkpointer=memory)
```

Key Benefits:

- Higher success rate
- Better fault tolerance
- Consistent results
- Scalable for production
- Easy debugging & monitoring

SUBTOPIC: LangGraph Debugging & Observability1. Graph Visualization

- Helps you see the structure of your LangGraph workflow (nodes, edges, conditions).
- Useful for understanding complex graphs and debugging routing issues.
- Can be viewed in LangSmith or using built-in visualization tools.

Example Visualizing a simple graphWhat you can see:

- Nodes (agents/tools)
- Edges (connections)
- Conditions
- Parallel branches
- Entry & exit points

2. Execution Traces

- Show the step-by-step execution of the graph run.
- Includes which node ran, input/output data, and timestamps.
- Helps find where things went wrong or took longer.

Example: Trace view (LangSmith)

Step	Node	Input	Output	Time
1	Input_node	-	{query: "..."}	12:00:01
2	agent	{query: ...}	{tool: search}	12:00:02
3	tool_node	{tool: search}	{result: ...}	12:00:05
4	agent	{result: ...}	{final: ...}	12:00:06
5	end	{final: ...}	-	12:00:07

→ Each step shows node name, input, output and execution time.

3. Input Inspection

- Lets you view the current state of the graph (values of state variables).
- Useful for checking how data of passed between nodes.
- Helps debug incorrect state updates or missing values.

Example Inspecting state in LangSmith

```
{
  "messages": [ ... ],
  "user_name": "Alice",
  "intermediate_result": "..."}

```

Key things to check:

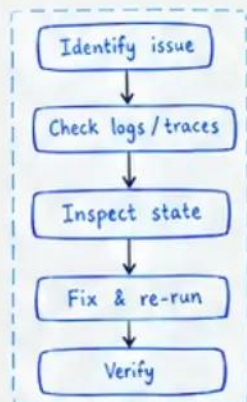
- State values
- Keys present
- Data types
- Unexpected changes

4. Debugging Workflows

- Use various useful debugging approach to find and fix issues.
- Combine logs, traces, state inspection and visualization.
- Re-run specific steps after making changes.

Typical debugging steps:

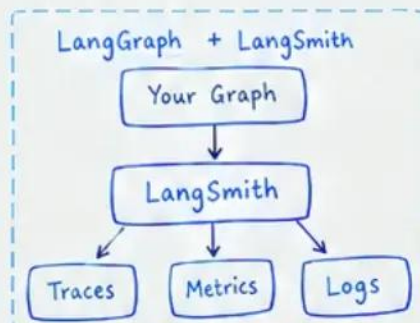
- 1 Check graph visualization (structure).
- 2 Look at execution traces (where it failed).
- 3 Inspect state (input/output values).
- 4 Fix the issue (logic, data, routing).
- 5 Re-run and verify.

5. LangGraph Integration

- LangSmith provides built-in debugging and observability for LangGraph.
- Lets you track runs, view traces, inspect state and evaluate performance.
- You can log custom data and set alerts.

Key features:

- Run tracking
- Graph visualization
- Execution traces
- State inspection
- Evaluation & feedback

6. Monitoring Agents

- Helps track logging latency, latency, and errors in production.
- Useful for single-agent or multi-agent workflows.
- You can set alerts for failures or unusual behavior.

Example: Monitoring with LangSmith★ Important:

Monitoring helps you build reliable, production-ready agents.

★ Key Takeaways:

- Use visualization, traces and state inspection to debug.
- LangSmith gives you powerful observability tools.
- Set up monitoring and alerts for production.
- Always verify after making changes.